

1 Secure Multi-Party Computation

Secure Multi-Party Computation (MPC) is a cryptographic framework that enables a collection of mutually distrustful parties to jointly compute a function over their private inputs without revealing those inputs to one another. The central objective is to obtain the desired computational result while preserving the privacy of the individual inputs.

Suppose there are n parties $P_1, P_2, \dots, P_n$, where party P_i holds a private input x_i . The parties wish to jointly compute $y = f(x_1, x_2, \dots, x_n)$ without revealing their private inputs beyond what can be inferred from the prescribed output.

The fundamental objective of MPC is therefore to compute $f(x_1, \dots, x_n)$ while keeping $x_1, \dots, x_n$ private.

1.1 Motivation for MPC

Consider a setting in which two hospitals wish to determine the total number of patients satisfying a particular medical condition. Let x_1 and x_2 denote the respective counts of the two hospitals. They may wish to obtain $x_1 + x_2$ without revealing either count to the other hospital.

A direct approach would require each hospital to disclose its private count, which violates the privacy requirement. MPC provides an alternative in which the hospitals execute a cryptographic protocol that computes the desired function while keeping their individual inputs confidential.

The same principle applies to many practical applications, including privacy-preserving data analysis, financial computations, collaborative computation, private database queries, and secure outsourced computation.

1.2 Basic MPC System Model

In the general MPC setting, each party P_i possesses a private input x_i . The parties execute a protocol Π whose purpose is to evaluate a function $f : \mathcal{X}_1 \times \dots \times \mathcal{X}_n \rightarrow \mathcal{Y}$.

At the end of the protocol, the prescribed output $y = f(x_1, \dots, x_n)$ is obtained.

An MPC protocol should provide, depending on the security model, the following properties:

1. **Correctness:** The protocol produces the correct output.
2. **Input privacy:** A party should not learn the private input of another party beyond what is revealed by the prescribed output.
3. **Security against corrupted parties:** The protocol should remain secure even when a specified number of parties are compromised.
4. **Fairness and robustness:** Depending on the protocol and threat model, parties may be prevented from obtaining an output unless the protocol completes correctly, or the protocol may tolerate malicious behavior.

The exact security guarantees depend on the adversarial model and the particular MPC construction.

1.3 Important MPC Protocol Families

MPC protocols can be broadly classified according to how the computation is represented and evaluated. Two important approaches are *garbled-circuit-based MPC* and *secret-sharing-based MPC*.

- **Garbled Circuits:** Yao's garbled-circuit protocol provides a fundamental approach to secure two-party computation. The function to be computed is represented as a Boolean circuit, and the parties jointly evaluate the circuit without revealing their private inputs.
- **GMW Protocols:** The GMW framework extends secure circuit evaluation to multiple parties. It represents a Boolean circuit using secret sharing and evaluates the circuit gate by gate.

- **Secret-Sharing-Based MPC:** In this approach, each private value is represented by shares distributed among the participating parties. The parties perform the computation directly on these shares without reconstructing the underlying private values.

Several important MPC protocols are based on secret sharing. For example, the **BGW protocol** uses polynomial secret sharing to achieve information-theoretic security against an appropriate number of corrupted parties. The **SPDZ family** uses secret sharing together with additional cryptographic techniques to provide efficient MPC with security against malicious adversaries.

Secret sharing is therefore one of the fundamental building blocks of modern MPC. Before studying complete MPC protocols, it is useful to understand how secrets can be distributed among multiple parties and how computations can be performed over the resulting shares.

2 Secret Sharing

Secret sharing is a cryptographic technique for distributing a secret among a set of participants such that only an authorized subset of participants can reconstruct the secret. The individual pieces distributed to the participants are called *shares*.

Let $\mathcal{S}$ denote the secret space and let $\mathcal{X}_1, \dots, \mathcal{X}_n$ denote the share spaces of participants $P_1, \dots, P_n$. A secret-sharing scheme consists of a randomized sharing algorithm Share and a reconstruction algorithm Rec .

For a secret $s \in \mathcal{S}$, the sharing algorithm generates shares

$$(x_1, \dots, x_n) \leftarrow \text{Share}(s),$$

where share x_i is distributed to participant P_i . For an authorized set $A \subseteq \{1, \dots, n\}$, reconstruction satisfies

$$\text{Rec}(\{x_i : i \in A\}) = s.$$

The scheme should also provide secrecy against unauthorized sets. If B is an unauthorized set of participants, then the shares $\{x_i : i \in B\}$ should reveal no information about the secret. Informally, the distribution of these shares should be independent of the secret.

Thus, a secret-sharing scheme has two fundamental requirements:

1. **Correctness:** An authorized set of shares can reconstruct the secret.
2. **Secrecy:** An unauthorized set of shares obtains no information about the secret.

3 Additive Secret Sharing

Additive secret sharing is a simple secret-sharing technique in which a secret is represented as the sum of multiple shares. It is particularly useful in MPC because addition and multiplication by public constants can be performed directly on the shares without reconstructing the underlying secret.

Let $\mathbb{F}$ be the underlying arithmetic domain and let $s \in \mathbb{F}$ denote the secret.

3.1 Two-Party Additive Sharing

Consider two parties P_1 and P_2 . To share the secret s , the dealer samples a uniformly random value $x_1 \xleftarrow{\$} \mathbb{F}$ and computes the second share as $x_2 = s - x_1$. The dealer gives x_1 to P_1 and x_2 to P_2 .

The secret is reconstructed by adding the two shares:

$$\text{Rec}(x_1, x_2) = x_1 + x_2 = s.$$

Since x_1 is uniformly random and independent of s , a party holding only x_1 obtains no information about the secret. Similarly, x_2 is also uniformly distributed over $\mathbb{F}$. Thus, either individual share is statistically independent of s , while the pair (x_1, x_2) uniquely determines the secret.

3.2 Generalization to n Parties

The construction naturally extends to n parties $P_1, \dots, P_n$. The dealer samples $x_1, \dots, x_{n-1} \xleftarrow{\$} \mathbb{F}$ independently and uniformly, and computes the final share as $x_n = s - \sum_{i=1}^{n-1} x_i$.

The dealer distributes x_i to P_i . The secret is reconstructed by adding all n shares. Thus, the scheme is an n -out-of- n secret-sharing scheme: all n shares are required for reconstruction, while any collection of fewer than n shares reveals no information about s .

3.3 Arithmetic Domain

Additive secret sharing can be defined over any suitable algebraic domain in which addition and subtraction are well-defined. In MPC, the underlying domain is commonly a finite field $\mathbb{F}_q$ or a finite ring such as $\mathbb{Z}_{2^\ell}$.

For example, when computation is performed modulo 2^{256} , the secret and shares belong to $\mathbb{Z}_{2^{256}}$, and reconstruction is performed modulo 2^{256} .

The choice of arithmetic domain is important when designing an MPC protocol. In particular, $\mathbb{Z}_{2^{256}}$ should not be confused with the finite field $\mathbb{F}_{2^{256}}$. Although both contain 2^{256} elements, they have different algebraic structures. In particular, $\mathbb{Z}_{2^{256}}$ is not a field because not every nonzero element has a multiplicative inverse, whereas $\mathbb{F}_{2^{256}}$ is a finite field.

3.4 Linear Computation over Additive Shares

An important property of additive secret sharing is that certain computations can be performed directly on the shares.

Suppose x and y are additively shared among n parties, where P_i holds shares x_i and y_i . Each party can locally compute $z_i = x_i + y_i$. The resulting values form an additive sharing of $x + y$.

More generally, addition and multiplication by a public constant can be performed locally on additive shares. Multiplication of two independently shared private values is more challenging and requires additional MPC techniques, such as Beaver multiplication triples.

4 Threshold Secret Sharing

In an n -out-of- n additive secret-sharing scheme, all n participants are required to reconstruct the secret. This is restrictive in practical systems because the failure or unavailability of even a single participant prevents reconstruction.

A more flexible construction is a *threshold secret-sharing* scheme.

(t, n) -Threshold Secret Sharing

Let $\mathcal{P} = \{P_1, \dots, P_n\}$ be a set of n participants and let $\mathcal{S}$ be the secret space. A (t, n) -threshold secret-sharing (TSS) scheme consists of a randomized sharing algorithm Share and a reconstruction algorithm Rec . For a secret $s \in \mathcal{S}$, the sharing algorithm generates shares

$$(x_1, \dots, x_n) \leftarrow \text{Share}(s),$$

where x_i is distributed to participant P_i .

The scheme satisfies the following properties:

1. **Correctness:** For every subset $A \subseteq \{1, \dots, n\}$ satisfying $|A| \geq t$, the secret can be reconstructed as $\text{Rec}(\{x_i : i \in A\}) = s$.
2. **Perfect Secrecy:** For every subset $B \subseteq \{1, \dots, n\}$ satisfying $|B| < t$, the shares held by B reveal no information about s . Formally, for any two secrets $s, s' \in \mathcal{S}$, the distributions of the corresponding shares are identical:

$$\{x_i : i \in B\} \mid s \stackrel{d}{=} \{x_i : i \in B\} \mid s'.$$

Here, t is called the *threshold* and n is the total number of participants. Thus, any t out of n participants can reconstruct the secret, while any coalition of fewer than t participants obtains no information about it.

The central challenge in threshold secret sharing is therefore to construct shares such that the secret is recoverable from any sufficiently large subset while remaining information-theoretically hidden from every unauthorized subset.

4.1 Polynomial Interpolation

The use of polynomials in threshold secret sharing is motivated by a simple geometric observation.

Consider a two-dimensional setting. A single point does not uniquely determine a line, since infinitely many straight lines can pass through one point. In contrast, two distinct points uniquely determine a straight line. This suggests a natural way to construct a $(2, n)$ -threshold secret-sharing scheme: represent the secret as a point on a line, distribute different points on the line as shares, and allow any two participants to reconstruct the line. Once the line is reconstructed, the secret can be recovered from a distinguished point on the line, such as its y -intercept $f(0)$.

This idea generalizes naturally to higher-degree polynomials. A polynomial of degree one is uniquely determined by any two distinct points, while a polynomial of degree two is uniquely determined by any three distinct points.

More generally, a polynomial of degree at most $t - 1$ is uniquely determined by any t distinct points. Therefore, if the secret is embedded as a specific value of the polynomial, such as $s = f(0)$, then any t participants can reconstruct the polynomial and recover the secret, whereas fewer than t points are insufficient to uniquely determine the polynomial.

This fundamental property of polynomial interpolation provides the mathematical foundation for Shamir's threshold secret-sharing scheme. The following section formalizes this construction.

5 Shamir's Threshold Secret Sharing

The polynomial interpolation idea leads directly to Shamir's threshold secret-sharing scheme. The scheme constructs a (t, n) -threshold secret-sharing scheme by representing the secret as a point on a randomly chosen polynomial over a finite field.

Let $\mathbb{F}$ be a sufficiently large finite field and let the secret be $s \in \mathbb{F}$. The dealer constructs a polynomial of degree at most $t - 1$:

$$f(x) = s + a_1x + a_2x^2 + \cdots + a_{t-1}x^{t-1},$$

where $a_1, \dots, a_{t-1} \xleftarrow{\$} \mathbb{F}$ are chosen independently and uniformly at random.

The secret is embedded as the constant term, so $f(0) = s$.

The dealer selects n distinct nonzero elements $x_1, \dots, x_n \in \mathbb{F}$ as evaluation points. For each participant P_i , the dealer computes $y_i = f(x_i)$ and gives the share (x_i, y_i) to P_i .

Thus, the shares correspond to n distinct points on the randomly chosen polynomial. The randomness in the coefficients makes the polynomial unpredictable to any collection of fewer than t participants.

5.1 Reconstruction

Suppose a set of t participants provides their shares $(x_1, y_1), \dots, (x_t, y_t)$. Since $f(x)$ has degree at most $t - 1$, these t distinct points uniquely determine the polynomial.

The polynomial can be reconstructed using Lagrange interpolation:

$$f(x) = \sum_{i=1}^t y_i \prod_{\substack{1 \leq j \leq t \\ j \neq i}} \frac{x - x_j}{x_i - x_j}.$$

However, reconstructing the entire polynomial is not necessary if the objective is only to recover the secret. Since the secret is $f(0)$, it can be obtained directly as

$$s = \sum_{i=1}^t y_i \prod_{\substack{1 \leq j \leq t \\ j \neq i}} \frac{-x_j}{x_i - x_j}.$$

All additions, multiplications, and divisions are performed over the finite field $\mathbb{F}$. Since the evaluation points are distinct, $x_i - x_j \neq 0$ for $i \neq j$, and hence the required inverses exist in $\mathbb{F}$.

Therefore, any t participants can reconstruct the secret, and the particular choice of the t participants is irrelevant.

5.2 Security of Shamir's Threshold Secret Sharing

We now prove the secrecy property of Shamir's (t, n) -threshold secret-sharing scheme. In particular, we show that any coalition of fewer than t participants obtains no information about the secret.

Proof

Let $B \subseteq \{1, \dots, n\}$ be any coalition of participants such that $|B| = k < t$. Suppose these participants hold the shares

$$\{(x_i, y_i) : i \in B\},$$

where $y_i = f(x_i)$ and

$$f(x) = s + a_1x + \cdots + a_{t-1}x^{t-1}.$$

Since $k < t$, the adversary observes fewer than t points of a polynomial of degree at most $t - 1$. These points do not uniquely determine the polynomial.

To see this more formally, consider any candidate secret $s' \in \mathbb{F}$. There exists a polynomial

$$f'(x) = s' + a'_1x + \cdots + a'_{t-1}x^{t-1}$$

of degree at most $t - 1$ that passes through all the observed points and satisfies $f'(0) = s'$. Therefore, the same set of observed shares is consistent with every possible value of the secret.

Moreover, because the coefficients $a_1, \dots, a_{t-1}$ are chosen independently and uniformly at random, the distribution of the shares observed by B is identical for every possible secret. Thus, for any $s, s' \in \mathbb{F}$,

$$\Pr[\{x_i, y_i : i \in B\} \mid s] = \Pr[\{x_i, y_i : i \in B\} \mid s'].$$

Hence, the shares held by any coalition of fewer than t participants are statistically independent of the secret. Therefore, they reveal no information about s .

Consequently, Shamir's scheme provides perfect secrecy against any coalition of fewer than t participants, while any t participants can reconstruct the secret. This establishes the security properties of the (t, n) -threshold secret-sharing scheme. $\square$

6 Secret Sharing over Finite Fields

Shamir's threshold secret-sharing scheme is defined over a finite field $\mathbb{F}_q$. Working over a finite field provides exact arithmetic and ensures that every nonzero element has a multiplicative inverse, which is required for polynomial interpolation.

All polynomial operations, including addition, multiplication, and division, are therefore performed in $\mathbb{F}_q$. In particular, division is defined as multiplication by the corresponding field inverse, so $a/b = ab^{-1}$ for $b \neq 0$.

The evaluation points $x_1, \dots, x_n$ must be distinct. This ensures that $x_i - x_j \neq 0$ for $i \neq j$, and hence the inverses required in Lagrange interpolation exist. If the evaluation points are chosen to be distinct and nonzero, the field must contain at least $n + 1$ elements, that is, $q \geq n + 1$.

6.1 Conditions for Shamir Secret Sharing

A valid (t, n) Shamir secret-sharing scheme requires the following conditions:

1. The sharing polynomial has degree at most $t - 1$.
2. The coefficients $a_1, \dots, a_{t-1}$, other than the secret coefficient, are chosen independently and uniformly at random from $\mathbb{F}_q$.
3. The evaluation points $x_1, \dots, x_n$ are distinct and nonzero elements of $\mathbb{F}_q$.
4. The field contains sufficiently many elements to provide the required evaluation points, in particular $q \geq n + 1$ for distinct nonzero points.
5. Reconstruction is performed using at least t valid shares.

7 Information-Theoretic Security and Post-Quantum Security

Shamir's secret-sharing scheme provides information-theoretic secrecy against any adversary that obtains fewer than t shares. Its security does not depend on the computational difficulty of any mathematical problem or on the computational resources available to the adversary.

In particular, the secrecy of Shamir's scheme does not rely on assumptions such as integer factorization, discrete logarithms, elliptic-curve discrete logarithms, or lattice-based computational assumptions. Even an adversary with unlimited computational power cannot recover or distinguish the secret from the shares of fewer than t participants.

This property has an important consequence in the context of post-quantum cryptography. Quantum algorithms such as Shor's algorithm can efficiently solve the integer factorization and discrete logarithm problems, thereby breaking cryptosystems whose security relies on these computational assumptions. However, unlimited computational power, including quantum computation, does not invalidate the information-theoretic secrecy of Shamir's scheme.

Therefore, the secrecy guarantee of Shamir secret sharing is inherently resistant to quantum computational attacks. In this sense, Shamir secret sharing provides a quantum-independent secrecy guarantee.

This does not imply that every MPC protocol or system using secret sharing is automatically post-quantum secure. The overall security of such a system may additionally depend on computational primitives such as authentication, commitments, encryption, pseudorandomness, preprocessing, and other protocol components. The post-quantum statement here applies specifically to the information-theoretic secrecy provided by the secret-sharing mechanism itself.